formalmind

WETH9

Formal Verification Audit

Formal Mind

Table of Contents

1. Summary . . . **2**

2. Functions . . . **3**

3. Statements . . . **4**

3.1. S-01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call. . . . 4

3.2. S-02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one. . . . 4

3.3. S-03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself. . . . 5

3.4. S-04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down. . . . 6

3.5. S-05: Anyone holding a balance can always withdraw all of it (no call by another account can stop them), unless their own address rejects ETH sent with 2300 gas. . . . 6

3.6. S-06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount. . . . 7

3.7. S-08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`). . . . 8

4. Function properties . . . **9**

4.1. P-01: WETH9.approve (verified) . . . 9

4.2. P-02: WETH9.«» (verified) . . . 9

4.3. P-03: WETH9.deposit (verified) . . . 9

4.4. P-04: WETH9.withdraw (verified) . . . 10

4.5. P-05: WETH9.transfer (verified) . . . 10

4.6. P-06: WETH9.transferFrom (verified) . . . 10

4.7. P-07: WETH9.totalSupply (verified) . . . 11

4.8. P-S04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down. (verified) . . . 11

4.9. P-S03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself. (verified) . . . 12

4.10. P-S08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`). (verified) . . . 13

4.11. P-S01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call. (verified) . . . 13

4.12. P-S02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one. (verified) . . . 15

4.13. P-S05: Anyone holding a balance can always withdraw all of it (no call by another account

can stop them), unless their own address rejects ETH sent with 2300 gas. (verified) . . . 17
4.14. P-S06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount. (verified) . . . 19
5. Scope and limits . . . **22**

Contract: `src/WETH9.sol` (canonical Wrapped Ether, Solidity `>=0.4.22 <0.6`)
Repository: formalmind/weth9 at `2cd3283`
Method: Formal Mind's agent swarm, against a Lean 4 model generated from solc's AST by formal-ast; every theorem checked by Lean (no `sorry`, no axioms beyond Lean's own) and approved by a reviewer agent; the record is the proof ledger (`.formal/ledger.json`).
Auditor: Formal Mind

1. Summary

- **7 of 7 functions proved**: for each, when it reverts, what it writes and what it returns.
- **7 of 7 statements proved**: properties of the whole contract, including solvency.
- **0 findings.** No property was disproved.

Check it yourself: `cd formal && lake build`.

2. Functions

Function	Status	Theorems	Shows
WETH9.deposit	proved	WETH9.deposit_spec, WETH9.deposit_frame, WETH9.fallback_spec	returns, reverts, writes
WETH9.«»	proved	WETH9.fallback_spec, WETH9.fallback_frame	returns, reverts, writes
WETH9.withdraw	proved	WETH9.withdraw_reverts, WETH9.withdraw_spec	returns, reverts, writes
WETH9.totalSupply	proved	WETH9.totalSupply_spec	returns, reverts, writes
WETH9.approve	proved	WETH9.approve_spec, WETH9.approve_allowance	returns, reverts, writes
WETH9.transferFrom	proved	WETH9.Swarm.Cons.transferFrom_conserves, WETH9.Swarm.transfer_reverts, WETH9.Swarm.transfer_succeeds, WETH9.Swarm.TF.transferFrom_reverts_balance, WETH9.Swarm.TF.transferFrom_reverts_allowance, WETH9.Swarm.TF.transferFrom_succeeds_spend, WETH9.Swarm.TF.transferFrom_succeeds_nospend	returns, reverts, writes
WETH9.transfer	proved	WETH9.Swarm.transfer_reverts, WETH9.Swarm.transfer_succeeds	returns, reverts, writes

3. Statements

3.1. S-01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of `balanceOf[a]` over all addresses a is at most `address(this).balance`. It holds at deployment (both zero). It keeps holding after every successful call to `deposit`, the receive/fallback, `withdraw`, `transfer`, `transferFrom` and `approve`, including when `withdraw's ETH send re-enters the contract.

Assumes:

- ETH forced in without a call (selfdestruct, coinbase rewards) only raises the contract's balance, so it can only widen the gap. It is allowed, and the claim is ≤ rather than =.
- The sum of balances is taken over the naturals (unbounded), not in uint256 with wrap-around.
- Calls that revert leave the state unchanged, so they can't break the invariant.

Proved for each function:

- `WETH9.approve`: proved
- `WETH9.«»`: proved
- `WETH9.deposit`: proved
- `WETH9.withdraw`: proved
- `WETH9.transfer`: proved
- `WETH9.transferFrom`: proved

3.2. S-02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of `balanceOf` over all accounts is at most `totalSupply()` (the contract's ETH balance, `address(this).balance`). Every entry point (receive/fallback, deposit, withdraw, transfer, transferFrom, approve) keeps this true. Equality is not guaranteed: ETH that reaches the contract without calling it (selfdestruct to it, coinbase rewards, ETH sent before deployment) raises `totalSupply()` but credits no `balanceOf`. So the gap `totalSupply() -`

`sum(balanceOf)` is never negative and can be positive.

Assumes:

- totalSupply() returns address(this).balance
- Forced ETH is modelled as an environment step that raises the contract's balance and changes no storage
- The ETH that withdraw sends out leaves the contract's balance exactly by wad, and a reentrant call during that send also keeps the invariant
- No uint256 overflow of balanceOf entries (sum of balances is bounded by total ETH)

Proved for each function:

- `WETH9.approve`: proved
- `WETH9.«»`: proved
- `WETH9.deposit`: proved
- `WETH9.withdraw`: proved
- `WETH9.transfer`: proved
- `WETH9.transferFrom`: proved

3.3. S-03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself.

Status: proved (conservation).

Precisely: For WETH9.transfer and WETH9.transferFrom only: after any call that doesn't revert, the total of balanceOf over all addresses is the same as before. This also holds when the sender and the recipient are the same address. Reverted calls change nothing. deposit and withdraw are not covered.

Assumes:

- The sum is taken over every address (all 2^160 of them), as a mathematical total of balanceOf
- Only successful calls are considered; a reverting call leaves state unchanged
- transfer is treated as transferFrom(msg.sender, dst, wad), as WETH9 implements it

Proved for each function:

- `WETH9.transfer`: proved

- `WETH9.transferFrom`: proved

3.4. S-04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down.

Status: proved (function · WETH9.transferFrom).

Precisely: Whenever a caller successfully moves a nonzero amount out of someone else's balance with transferFrom, src must have approved that caller for at least that amount beforehand. The allowance then goes down by exactly the amount moved, except an allowance of type(uint256).max, which stays the same. The other functions that write balanceOf (deposit, withdraw, transfer) are taken to change only the caller's own balance, or to raise someone else's. That part is an assumption here and is not proved.

Assumes:

- deposit, withdraw and transfer never lower the balance of an account other than msg.sender (transfer delegates to transferFrom with src = msg.sender)
- Transfers to oneself (src == dst) and zero-amount transfers don't count as lowering a balance
- uint(-1) means type(uint256).max

Proved for each function:

- `WETH9.transferFrom`: proved

3.5. S-05: Anyone holding a balance can always withdraw all of it (no call by another account can stop them), unless their own address rejects ETH sent with 2300 gas.

Status: proved (sequence).

Precisely: Take any reachable state of WETH9 and any address A with balanceOf[A] = b. After any sequence of calls by other accounts (deposit, withdraw, transfer, transferFrom, approve), A can still call withdraw(balanceOf[A]) and it succeeds: A's balance drops to 0 and A receives that much ETH. Other accounts can't lower balanceOf[A] without A's approval, and they can't drain the contract's ETH below the total of all balances. The only allowed failure is A's own address reverting or running out of the 2300 gas when `transfer` sends it ETH. This rests on the fact that WETH9's ETH balance is always at least the sum of balanceOf (solvency).

Assumes:

- A has given no allowance, or 'another account' excludes spenders A has approved: an approved spender can move A's balance with transferFrom
- The call has enough gas, and the call depth limit is not reached
- Solvency: address(WETH9).balance >= sum of balanceOf in every reachable state. ETH forced in through selfdestruct only makes this easier to hold
- A's balance can grow through deposits or incoming transfers, and 'all of it' means A's balance at the moment A withdraws

Proved for each function:

- `WETH9.approve`: proved
- `WETH9.«»`: proved
- `WETH9.deposit`: proved
- `WETH9.withdraw`: proved
- `WETH9.transfer`: proved
- `WETH9.transferFrom`: proved

3.6. S-06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount.

Status: proved (invariant).

Precisely: In every reachable state of WETH9, the sum of balanceOf[a] over all addresses a plus the ETH forced into the contract (via selfdestruct or as a coinbase/block reward, which runs no code) equals address(this).balance, which is what totalSupply() returns. This must hold after every state-changing entry point (deposit and the receive/fallback path, withdraw, transfer, transferFrom, approve). If no ETH was ever forced in, the sum of balances is exactly totalSupply(). The forced amount is the total ETH received without going through deposit or the fallback.

Assumes:

- Forced ETH is modelled as ghost state: the running total of ETH that arrived without calling deposit or the fallback
- withdraw's ETH transfer to msg.sender either succeeds in full or reverts the whole call (re-entry during the send leaves the relation intact)

- The contract starts with zero balance and zero balanceOf everywhere, apart from ETH already forced in, which counts toward the forced amount

Proved for each function:

- `WETH9.approve`: proved
- `WETH9.«»`: proved
- `WETH9.deposit`: proved
- `WETH9.withdraw`: proved
- `WETH9.transfer`: proved
- `WETH9.transferFrom`: proved

3.7. S-08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`).

Status: proved (conservation).

Precisely: For any call to `WETH9.transfer(dst, wad)` or `WETH9.transferFrom(src, dst, wad)` that does not revert, every address other than `src` and `dst` has the same `balanceOf` after the call as before. For `transfer`, `src` is `msg.sender`. Nothing is claimed about `allowance`, about the new balances of `src` and `dst`, or about calls that revert.

Assumes:

- For transfer, src is msg.sender (transfer delegates to transferFrom(msg.sender, dst, wad)).
- Only successful (non-reverting) calls are covered; a revert leaves all state unchanged anyway.
- src == dst is allowed; then only that one account may change (it actually nets to zero).

Proved for each function:

- `WETH9.transfer`: proved
- `WETH9.transferFrom`: proved

4. Function properties

4.1. P-01: WETH9.approve (verified)

`WETH9.approve(guy, wad)` never reverts, for any caller, environment or storage, and always returns `true`. * `approve_spec`: the resulting state is the prior state with the contract's storage replaced by a copy in which only `allowance[msg.sender][guy]` is set to `wad`. * `approve_allowance`: afterwards `allowance[o][sp]` equals `wad` when `o = msg.sender` and `sp = guy`, and its previous value otherwise. `balanceOf` is unchanged.

Assumptions. For `approve_allowance` only: the storage lens satisfies `ext.get (ext.set w s) = s`. The `Approval` event is not modelled, so it is not covered.

4.2. P-02: WETH9.«» (verified)

- `fallback_spec`: `WETH9.«»` (the payable fallback) never reverts. For every environment and world `w`, it returns `.ok ((), w')`, where `w'` is `w` with `balanceOf[msg.sender]` set to `(balanceOf[msg.sender] + msg.value) mod 2^256` (unchecked, as in the source; the model adds no checked-overflow guard). All other storage is unchanged. It makes no external calls and returns nothing.
- `fallback_frame`: on success, `balanceOf[msg.sender]` holds the wrapped sum. `balanceOf[a]` is unchanged for every `a ≠ msg.sender`, and `allowance`, `name`, `symbol` and `decimals` are unchanged.

Assumptions. `fallback_frame` assumes the storage lens is lawful: `ext.get (ext.set w s) = s`. `fallback_spec` assumes nothing. The `Deposit` event is not modelled.

4.3. P-03: WETH9.deposit (verified)

- `WETH9.deposit` never reverts: for every caller, `msg.value` and starting state, it returns `()` in the state where `balanceOf[msg.sender]` is replaced by `(balanceOf[msg.sender] + msg.value) mod 2^256` (an unchecked `addw 256`) and nothing else is written (`deposit_spec`).
- On success, `balanceOf[msg.sender]` holds that sum. Every other `balanceOf` entry, `allowance`, `name`, `symbol` and `decimals` are unchanged (`deposit_frame`).

Assumptions. `deposit_frame` assumes the storage lens law `ext.get (ext.set w s) = s`. The emitted `Deposit` event and the contract's ETH balance are not modelled. The addition wraps mod 2^256 as in the Solidity (a wrap would need more than 2^256 wei, which cannot exist).

4.4. P-04: WETH9.withdraw (verified)

- `withdraw_reverts`: `WETH9.withdraw ext wad` reverts with `"require"` whenever `balanceOf[msg.sender] < wad`.
- `withdraw_spec`: when `wad ≤ balanceOf[msg.sender]`, `WETH9.withdraw` first writes only `balanceOf[msg.sender] := subw 256 (balanceOf[msg.sender]) wad` (all other storage fields and keys untouched), then makes exactly one low-level call to `msg.sender` with value `wad` and empty calldata, on the already-updated world (checks-effects-interactions order). The function returns `()` with the call's resulting world iff the call reports success; it reverts with `"transfer"` if the call returns false and propagates any revert of the call.
- `withdraw_sub_exact`: under the guard and `balanceOf[msg.sender] < 2^256`, the unchecked subtraction does not wrap: `subw 256 bal wad = bal - wad`.

Assumptions. The external call is modelled by the opaque `ext.lowCall`; nothing is assumed about its behaviour (it may re-enter and change state arbitrarily). `withdraw_sub_exact` assumes the stored balance is a uint256 value (< 2^256).

4.5. P-05: WETH9.transfer (verified)

- `transfer_reverts`: `WETH9.transfer dst wad` reverts with `require` whenever `balanceOf[msg.sender] < wad`.
- `transfer_succeeds`: when `wad ≤ balanceOf[msg.sender]`, `WETH9.transfer` returns `true`. Its only storage change is to `balanceOf`: first `balanceOf[msg.sender]` is decreased by `wad`, then `balanceOf[dst]` is increased by `wad`, both modulo 2^256 and with the second reading the already-debited map (so a self-transfer leaves the balance unchanged). `allowance` is never touched, because `src = msg.sender` skips the allowance branch. Taken together, the two theorems show that the balance guard is the only reason `transfer` can revert.

Assumptions. The storage accessor `ext` is a lawful lens: `get (set w s) = s` and `set (set w s) t = set w t`.

4.6. P-06: WETH9.transferFrom (verified)

- `WETH9.transferFrom` reverts with `require` when `balanceOf[src] < wad`.
- When the balance check passes, `src != msg.sender`, `allowance[src][msg.sender] != 2^256-1` and `allowance[src][msg.sender] < wad`, it reverts with `require`.
- When `src != msg.sender`, the allowance is finite and both checks pass, it returns `true`. It sets `allowance[src][msg.sender]` to the old value minus `wad`, then sets `balanceOf[src]` to the old value minus `wad`, then adds `wad` to `balanceOf[dst]`, all modulo 2^256. No other state changes.

- When `src == msg.sender` or the allowance is `2^256-1`, and the balance check passes, it returns `true`. Only `balanceOf` changes, with the same debit-then-credit update, and `allowance` is left as it was.
- Whenever it succeeds, the sum of `balanceOf` over any duplicate-free list of accounts containing `src` and `dst` stays the same (`transferFrom_conserves`). This also holds when `src == dst`. It requires `balanceOf[src] < 2^256` and, when `src != dst`, `balanceOf[dst] + wad < 2^256`.

Assumptions. The storage accessor `ext` is lawful: `get (set w s) = s` and `set (set w s) t = set w t`. These are needed only for the success theorems. The conservation theorem also takes as given that no value overflows: `balanceOf[src] < 2^256`, and `balanceOf[dst] + wad < 2^256` when `src != dst`. The `Transfer` event is not modelled.

4.7. P-07: WETH9.totalSupply (verified)

For every environment `env` and storage state `s`, `WETH9.totalSupply` never reverts, leaves storage unchanged, and returns `env.balance`, the contract's own ether balance (`address(this).balance`): `((WETH9.totalSupply ext).run env).run s = .ok (env.balance, s)`.

Assumptions. The model's `Env.balance` stands for `address(this).balance` during the call.

4.8. P-S04: No one can lower another account's balance except through an allowance that account gave them, and an allowance of uint(-1) is never spent down. (verified)

Whenever a caller successfully moves a nonzero amount out of someone else's balance with transferFrom, src must have approved that caller for at least that amount beforehand. The allowance then goes down by exactly the amount moved, except an allowance of type(uint256).max, which stays the same. The other functions that write balanceOf (deposit, withdraw, transfer) are taken to change only the caller's own balance, or to raise someone else's. That part is an assumption here and is not proved.

`S04.WETH9_transferFrom`: whenever `WETH9.transferFrom(src, dst, wad)` returns successfully with `src ≠ msg.sender`, two things hold: * Before the call, `allowance[src][msg.sender] ≥ wad`. * After the call, `allowance[src][msg.sender]` equals its old value minus `wad`. If the old value was `type(uint256).max`, it is unchanged.

A third party can therefore debit `src` only up to an allowance `src` granted. Each successful debit spends exactly `wad` of that allowance, and an unlimited allowance is never spent down.

Assumptions. deposit, withdraw and transfer never lower the balance of an account other than msg.sender (transfer delegates to transferFrom with src = msg.sender). Transfers to oneself (src == dst)

and zero-amount transfers don't count as lowering a balance. uint(-1) means type(uint256).max. The storage accessor `ext` is lawful: get after set returns what was set, and a second set overwrites the first. `wad` and the stored allowance are below 2^256, which ABI decoding and uint256 storage guarantee. The hypotheses `src ≠ dst` and `wad > 0` from the statement are in the theorem but not needed by the proof. That deposit, withdraw and transfer never lower a balance other than the caller's is assumed, as the maintainer agreed, and is not proved here.

4.9. P-S03: transfer and transferFrom conserve the sum of balances, including a transfer to yourself. (verified)

For WETH9.transfer and WETH9.transferFrom only: after any call that doesn't revert, the total of balanceOf over all addresses is the same as before. This also holds when the sender and the recipient are the same address. Reverted calls change nothing. deposit and withdraw are not covered.

`WETH9.transfer`

`WETH9.transfer` conserves total WETH: for any call that returns `.ok (r, w')`, the sum of `balanceOf a` over every address `a < 2^160` (`S03T.total balanceOf (2^160)`) is the same in `w'` as in `w`. This includes the self-transfer case `dst = msg.sender`, where the balance mapping comes back exactly as it was (debit `b − wad`, then credit `(b − wad) + wad = b`, with no wraparound). When `dst ≠ msg.sender`, the sender loses exactly `wad` and `dst` gains exactly `wad`. Reverting calls (`balanceOf[msg.sender] < wad`) produce no post-state. Supporting lemmas: `S03T.total_upd` (updating one entry k < n replaces `f k` by `v` in the total) and `S03T.total_congr`.

`WETH9.transferFrom`

`WETH9.transferFrom` conserves the sum of `balanceOf` over every address (`List.range (2^160)`): on any successful call the total after equals the total before. This holds whether the allowance is spent or left alone (`src = msg.sender`, or allowance `2^256 - 1`). It also holds when `src = dst`, where the debit and credit hit the same entry and leave the balance unchanged.

Assumptions. The sum is taken over every address (all 2^160 of them), as a mathematical total of balanceOf. Only successful calls are considered; a reverting call leaves state unchanged. transfer is treated as transferFrom(msg.sender, dst, wad), as WETH9 implements it. The storage accessor `ext` satisfies get-after-set and set-after-set. `msg.sender` and `dst` are addresses (< 2^160). Every `balanceOf` entry is a uint256 (< 2^256). When `dst ≠ msg.sender`, crediting `dst` doesn't overflow: `balanceOf[dst] + wad < 2^256`. This always holds while total supply is below 2^256, as it is for WETH, which is backed by ETH. Without it, the wrapping (unchecked) `=` would only conserve the sum mod 2^256. `ext.get`/`ext.set` form a lens: get-after-set returns the stored state, and set-after-set keeps only the last write. `src` and `dst` are below 2^160. `balanceOf[src]` is below 2^256. When `src ≠ dst`, `balanceOf[dst] + wad < 2^256`, so the unchecked `=` doesn't wrap. The theorem assumes this rather than deriving it. it would follow from a total-supply bound, which isn't

proved here. Only successful calls are considered.

4.10. P-S08: `WETH9.transfer` and `WETH9.transferFrom` change `balanceOf` only at `src` and `dst`: every other account's `balanceOf` is unchanged (and `allowance` changes only at `[src][msg.sender]` in `transferFrom`). (verified)

For any call to `WETH9.transfer(dst, wad)` or `WETH9.transferFrom(src, dst, wad)` that does not revert, every address other than `src` and `dst` has the same `balanceOf` after the call as before. For `transfer`, `src` is `msg.sender`. Nothing is claimed about `allowance`, about the new balances of `src` and `dst`, or about calls that revert.

`WETH9.transfer`

`WETH9.transfer(dst, wad)` changes `balanceOf` only at `msg.sender` and `dst`: for every successful (non-reverting) call and every address `a` with `a ≠ msg.sender` and `a ≠ dst`, `balanceOf[a]` after the call equals `balanceOf[a]` before it.

`WETH9.transferFrom`

For every call to `WETH9.transferFrom(src, dst, wad)` that does not revert, every address `a` with `a ≠ src` and `a ≠ dst` has the same `balanceOf[a]` after the call as before.

Assumptions. For transfer, src is msg.sender (transfer delegates to transferFrom(msg.sender, dst, wad)). Only successful (non-reverting) calls are covered; a revert leaves all state unchanged anyway. src == dst is allowed; then only that one account may change (it actually nets to zero). The storage accessor `ext` behaves like a store: `get (set w s) = s` and `set (set w s) t = set w t`. The storage lens `ext` is lawful: reading after a write returns the written state, and a second write replaces the first.

4.11. P-S01: Solvency: the sum of all balances never exceeds the ETH the contract holds, after any call. (verified)

In every reachable state of WETH9, the sum of `balanceOf[a]` over all addresses a is at most `address(this).balance`. It holds at deployment (both zero). It keeps holding after every successful call to `deposit`, the receive/fallback, `withdraw`, `transfer`, `transferFrom` and `approve`, including when `withdraw's ETH send re-enters the contract.

`WETH9.approve`

`WETH9.approve guy wad` preserves solvency (S-01). The call always succeeds and returns `true`. If, before the call, `▒_{a▒S} balanceOf a ≤ address(this).balance` holds for every list `S` of distinct addresses, then it holds afterwards too. The call writes only `allowance[msg.sender][guy]`, so

balanceOf and the contract's ETH balance stay the same.

WETH9.«»

WETH9.«» (the fallback, which runs deposit) preserves solvency. Suppose that before the call, for every duplicate-free list of addresses S, Σ_{a≣S} balanceOf[a] ≤ address(this).balance − msg.value (the contract's ETH before msg.value arrived), and msg.value ≤ address(this).balance. Then the call succeeds. In the resulting state, Σ_{a≣S} balanceOf[a] ≤ address(this).balance for every such S. The only write is balanceOf[msg.sender], which is set to (old + msg.value) mod 2^256. That value is at most old + msg.value, so the sum grows by at most msg.value.

WETH9.deposit

WETH9.deposit preserves solvency (S-01). Take any state where, for every list of distinct addresses, the sum of balanceOf is at most the contract's ETH balance from before msg.value was credited (env.balance - env.value). In that state deposit succeeds, and in the resulting state the sum of balanceOf over any list of distinct addresses is at most address(this).balance (env.balance). The unchecked balanceOf[msg.sender] += msg.value writes (b + v) mod 2^256 ≤ b + v, so the sum rises by at most msg.value. This holds even when the addition wraps around.

WETH9.withdraw

S01.WETH9_withdraw: for any successful call WETH9.withdraw wad taking world w to w', if Σ_{a≣S} balanceOf[a] ≤ ethBal(w) holds for every finite set S of distinct addresses, then Σ_{a≣S} balanceOf[a] ≤ ethBal(w') holds for every such S. * The guard balanceOf[sender] ≥ wad and a uint256 balance make the unchecked subtraction exact. After the write, Σ_S balanceOf + wad ≤ Σ_{S≣{sender}} old balanceOf ≤ ethBal (S01.withdraw_sum_upd, S01.withdraw_sum_upd_notin). So the contract keeps at least wad of slack before the ETH is sent. * The reverting paths (failed guard, failed or reverting send) produce no .ok result, so they are excluded.

WETH9.transfer

For WETH9.transfer: if the sum of balanceOf[a] over every address a (a < 2^160) is at most address(this).balance before a successful call, it is still at most that after. A successful transfer leaves the total of balances unchanged (theorem S03.WETH9_transfer, which also covers dst = msg.sender), and it moves no ETH, so the contract's balance is the same before and after.

WETH9.transferFrom

If the sum of balanceOf[a] over all 2^160 addresses (taken over the naturals) is at most address(this).balance before a successful call to WETH9.transferFrom, the same bound holds afterwards. This covers any src, dst and wad, whether the allowance is spent or not (caller is src, or allowance is 2^256−1), and src = dst. A successful transferFrom leaves the total of balances unchanged. The call is nonpayable and sends no ETH, so the contract's ETH balance does not change.

Assumptions. ETH forced in without a call (selfdestruct, coinbase rewards) only raises the contract's balance, so it can only widen the gap. It is allowed, and the claim is ≤ rather than =. The sum of balances is taken over the naturals (unbounded), not in uint256 with wrap-around. Calls that revert leave the state unchanged, so they can't break the invariant. Reading the storage record back after writing it returns what was written (`ext.get (ext.set w s) = s`). Balances are summed as unbounded naturals, with no uint256 wrap-around. The storage accessor satisfies get-after-set (`ext.get (ext.set w s) = s`). `env.balance` already includes `msg.value`, as in the EVM. Sums are taken over the naturals, over any finite duplicate-free set of addresses. `ext.get (ext.set w s) = s`. `env.balance` is the contract's balance during the call, already credited with `msg.value` (so `msg.value ≤ env.balance`). Sums are taken in ≣ over any finite list of distinct addresses. - `ethBal : □ → Nat` models the contract's ETH balance in a world state, and storage writes (`ext.set`) do not change it. - `ext.get (ext.set w s) = s`. - `balanceOf[msg.sender] < 2^256` (uint256 storage). - The low-level call behind `msg.sender.transfer(wad)` (`ext.lowCall`) is taken as given: if it succeeds from a world where the sum of balanceOf over any set of distinct addresses, plus `wad`, is at most `ethBal`, then in the resulting world every such sum is at most `ethBal`. This covers sending `wad` ETH out and any re-entrant calls. Justifying it relies on the S-01 preservation theorems for the other entry points, by an argument over call depth. The storage accessors behave correctly (get after set returns what was set. a second set overwrites the first). The sender and `dst` are 160-bit addresses. Every stored balance is below 2^256. When `dst ≠ msg.sender`, `balanceOf[dst] + wad` does not overflow uint256. The sum is taken over the naturals. `ext.get`/`ext.set` behave as a lens on WETH9's storage (get-after-set, set-after-set). `src` and `dst` are below 2^160. The stored `balanceOf[src]` is below 2^256. When `src ≠ dst`, `balanceOf[dst] + wad` is below 2^256, so the unchecked `+=` does not wrap. This follows from the invariant whenever the ETH balance is below 2^256. The same environment (ETH balance) holds before and after the call.

4.12. P-S02: totalSupply() can be larger than the sum of balances: ETH forced in (selfdestruct, coinbase) is counted but owned by no one. (verified)

In every reachable state of WETH9, the sum of `balanceOf` over all accounts is at most `totalSupply()` (the contract's ETH balance, `address(this).balance`). Every entry point (receive/fallback, deposit, withdraw, transfer, transferFrom, approve) keeps this true. Equality is not guaranteed: ETH that reaches the contract without calling it (selfdestruct to it, coinbase rewards, ETH sent before deployment) raises `totalSupply()` but credits no `balanceOf`. So the gap `totalSupply() - sum(balanceOf)` is never negative and can be positive.

`WETH9.approve`

`WETH9.approve` preserves S-02. The call always succeeds and returns `true`. Its only storage write is `allowance[msg.sender][guy] = wad`, so every `balanceOf` entry is unchanged, and the contract's ETH balance (`totalSupply()`, modelled as `env.balance`) is the same before and after. So if `□_{a□S} balanceOf a ≤ totalSupply()` holds before the call for every duplicate-free list of accounts `S`, it holds

after.

`WETH9.«»`

`WETH9.«»` (the fallback, which runs `deposit`) keeps S-02: if for every duplicate-free list of accounts S, Σ_{a∈S} `balanceOf[a]` ≤ `address(this).balance − msg.value` before the call, then the call succeeds and afterwards Σ_{a∈S} `balanceOf[a]` ≤ `address(this).balance` for every such S. Only `balanceOf[msg.sender]` changes, by at most `msg.value` (the wrapped addition never goes above the exact sum).

`WETH9.deposit`

`WETH9.deposit` preserves the invariant Σ `balanceOf` ≤ `totalSupply()` (= `address(this).balance`). Suppose that before `msg.value` arrives, the sum of `balanceOf` over every finite list of distinct accounts is at most `balance - msg.value`. Then the call succeeds, and afterwards that sum is at most `balance`. Crediting `msg.sender` with the unchecked `addw 256 (b + v)` gives at most `b + v`, so the sum rises by at most `msg.value` while the ETH balance rises by exactly `msg.value`.

`WETH9.withdraw`

`WETH9.withdraw` keeps the S-02 invariant. Suppose that before the call, Σ `balanceOf` over every finite list of distinct accounts is at most the contract's ETH balance (`totalSupply()`), and the call succeeds. Then the same bound holds after the call. The guard `balanceOf[msg.sender] >= wad` makes the subtraction exact, so the storage write lowers Σ `balanceOf` by `wad` and leaves Σ `balanceOf` + `wad` ≤ ETH balance before the ETH is sent.

`WETH9.transfer`

`WETH9.transfer` preserves S-02: if ∑_{a < 2^160} `balanceOf a ≤ address(this).balance` holds before a successful call, it holds after. A successful `transfer` leaves the sum of all balances unchanged, including the self-transfer case `dst = msg.sender` (S03.WETH9_transfer). The function is nonpayable and sends no ETH, so the contract's balance is unchanged too.

`WETH9.transferFrom`

`WETH9.transferFrom` preserves the solvency bound: if the sum of `balanceOf` over all 2^160 addresses is at most `address(this).balance` (`totalSupply()`) before a successful call, it is still at most that after. Either allowance branch leaves the sum of balances unchanged, and the call moves no ETH, so the gap `totalSupply() −` ∑ `balanceOf` stays the same.

Assumptions. totalSupply() returns address(this).balance. Forced ETH is modelled as an environment step that raises the contract's balance and changes no storage. The ETH that withdraw sends out leaves the contract's balance exactly by wad, and a reentrant call during that send also keeps the invariant. No uint256 overflow of balanceOf entries (sum of balances is bounded by total ETH). The storage accessor

reads back what was written (`ext.get (ext.set w s) = s`). `totalSupply()` is modelled as `env.balance`, which the call environment fixes. the nonpayable `msg.value` check is not modelled. `env.balance` is the contract's ETH balance after `msg.value` is credited (so `msg.value ≤ balance`), and `totalSupply()` = `address(this).balance`. The storage extension's `get` reads back what `set` wrote. Forced ETH only raises the balance and changes no storage, so it keeps the invariant trivially. Reading the state after writing it returns what was written (`ext.get (ext.set w s) = s`). `msg.value ≤ balance`, where the balance seen during the call already includes `msg.value`. "Sum over all accounts" means the sum over every finite list of distinct addresses. `ethBal` is the contract's ETH balance (`totalSupply()`), and storage writes don't change it. `ext.get` after `ext.set` returns the stored state. The low-level send of `wad` to msg.sender, including any reentrant calls, takes any state with Σ balanceOf + wad ≤ ethBal to one with Σ balanceOf ≤ ethBal. The sender's balance is below 2^256. `msg.sender` and `dst` are addresses (< 2^160). Every balance is < 2^256, and `balanceOf[dst] + wad` does not overflow when `dst ≠ msg.sender`. The storage accessors satisfy get-after-set and set-after-set. `address(this).balance` is `env.balance`, which this call does not change. `totalSupply()` is modeled as `e.balance`, which this nonpayable call does not change. Storage accessors are lawful: a read after a write returns what was written, and a later write replaces an earlier one. `src` and `dst` are 160-bit addresses. `balanceOf[src] < 2^256`, and if `src ≠ dst` then `balanceOf[dst] + wad < 2^256` (no uint256 wrap).

4.13. P-S05: Anyone holding a balance can always withdraw all of it (no call by another account can stop them), unless their own address rejects ETH sent with 2300 gas. (verified)

Take any reachable state of WETH9 and any address A with balanceOf[A] = b. After any sequence of calls by other accounts (deposit, withdraw, transfer, transferFrom, approve), A can still call withdraw(balanceOf[A]) and it succeeds: A's balance drops to 0 and A receives that much ETH. Other accounts can't lower balanceOf[A] without A's approval, and they can't drain the contract's ETH below the total of all balances. The only allowed failure is A's own address reverting or running out of the 2300 gas when `transfer` sends it ETH. This rests on the fact that WETH9's ETH balance is always at least the sum of balanceOf (solvency).

`WETH9.approve`

For any address `A` and any call to `WETH9.approve(guy, wad)` whose `msg.sender` is not `A`, the call: * always succeeds and returns `true`; * leaves the whole `balanceOf` mapping unchanged, so `A's balance and the sum of all balances stay the same; * leaves `allowance[A][·]` unchanged, so it gives no new account the right to spend `A's balance.

`approve` is nonpayable and makes no external calls, so the contract's ETH balance does not change either. Solvency and `A's ability to `withdraw(balanceOf[A])` therefore still hold after another account calls `approve`.

`WETH9.«»`

Take a call to the fallback `WETH9.«»` (which runs `deposit`) by any sender other than an address A. Assume the state is solvent before the call: for every duplicate-free list of addresses, the sum of their `balanceOf` is at most the contract's ETH balance excluding this call's `msg.value`. Then: * the call succeeds; * `balanceOf[A]` is unchanged; * every `allowance` entry is unchanged; * solvency still holds afterwards: the sum of `balanceOf` over any duplicate-free list of addresses is at most `address(this).balance`.

`WETH9.deposit`

`WETH9.deposit` called by any account other than `A` never reverts. On success: * `balanceOf[A]` is unchanged; * every allowance is unchanged; * solvency holds afterwards: for every duplicate-free list of addresses, the sum of `balanceOf` is at most the contract's ETH balance, given that it was at most `balance − msg.value` before.

So a deposit by another account cannot lower `A's balance or make the contract unable to pay `A` out.

`WETH9.withdraw`

For any account `A` and any successful call `WETH9.withdraw(wad)` whose `msg.sender ≠ A`: * `balanceOf[A]` afterwards is at least what it was before. The function's only storage write is to `balanceOf[msg.sender]`. * Solvency holds afterwards: for every set of distinct addresses, the sum of their `balanceOf` is at most the contract's ETH balance.

So a withdraw by another account lowers neither `A's withdrawable balance nor the ETH that backs it.

`WETH9.transfer`

For any account `A` and any call `WETH9.transfer(dst, wad)` made by a sender other than `A` that completes successfully, `balanceOf[A]` afterwards is at least its value before, and `allowance[A]` (every spender's entry) is unchanged. A call by another account to `transfer` therefore cannot lower `A's balance or change `A's approvals.

`WETH9.transferFrom`

`WETH9.transferFrom` called by an account other than `A` that `A` has not approved (`allowance[A][msg.sender] = 0`) never lowers `balanceOf[A]`: on every successful run, `balanceOf'[A] ≥ balanceOf[A]`. * If `src = A`, the allowance guard (`allowance[A][sender] ≥ wad`, 0 ≠ 2^256−1) leaves `wad = 0` as the only value that succeeds, so A's balance is unchanged. * If `src ≠ A`, A's balance is either untouched or credited by `wad` (when `dst = A`). So no third party can lower A's balance through `transferFrom`, and A's later `withdraw` of their full balance is not blocked by such a call.

Assumptions. A has given no allowance, or 'another account' excludes spenders A has approved: an approved spender can move A's balance with transferFrom. The call has enough gas, and the call depth

limit is not reached. Solvency: address(WETH9).balance >= sum of balanceOf in every reachable state. ETH forced in through selfdestruct only makes this easier to hold. A's balance can grow through deposits or incoming transfers, and 'all of it' means A's balance at the moment A withdraws. The storage accessor satisfies `ext.get (ext.set w s) = s`. The model does not track ETH. That the contract's ETH balance is unchanged comes from the Solidity (`approve` is nonpayable and makes no external calls), not from the Lean theorem. Storage read-after-write: `ext.get (ext.set w s) = s`. Before the call, the sum of balances over any duplicate-free address list is at most `env.balance - env.value`, and `env.value ≤ env.balance` (the environment's balance already includes `msg.value`). The sender is not A. `ext.get (ext.set w s) = s`, i.e. the storage accessor reads back what it wrote. `msg.value ≤ env.balance`, i.e. the incoming ETH is already counted in the contract's balance. Solvency held before the call, measured against the balance without `msg.value`. Storage writes do not change the contract's ETH balance, and `ext.get (ext.set w s) = s`. The caller's balance is below 2^256. The pre-state is solvent. The external ETH send to `msg.sender` is treated as a black box, and so is anything it re-enters. It is assumed not to lower `balanceOf[A]`. If it starts with Σ balanceOf + wad ≤ ETH, it is assumed to end solvent. This is the inductive hypothesis for the other functions of the claim. The storage accessor laws `ext.get (ext.set w s) = s` and `ext.set (ext.set w s) t = ext.set w t`. `balanceOf[A] + wad < 2^256`: since `wad` is at most the sender's balance, this follows from solvency (the sum of balances is at most the contract's ETH, which is far below 2^256). The proof covers only calls that succeed. a call that reverts leaves no state change. `ext.get`/`ext.set` behave as a lens (get after set returns the set state. set after set keeps the last). `balanceOf[A] < 2^256`, and crediting `wad` to A does not overflow (`balanceOf[A] + wad < 2^256`), which holds because the total supply is bounded by the ETH WETH9 holds. A has given the caller no allowance (an approved spender may move A's funds by design).

4.14. P-S06: The sum of `WETH9.balanceOf` over all accounts equals `totalSupply()`, i.e. `address(this).balance`, apart from ETH forced in (selfdestruct or coinbase): without forced ETH it is exact equality, and with it the sum is lower by exactly the forced amount. (verified)

In every reachable state of WETH9, the sum of balanceOf[a] over all addresses a plus the ETH forced into the contract (via selfdestruct or as a coinbase/block reward, which runs no code) equals address(this).balance, which is what totalSupply() returns. This must hold after every state-changing entry point (deposit and the receive/fallback path, withdraw, transfer, transferFrom, approve). If no ETH was ever forced in, the sum of balances is exactly totalSupply(). The forced amount is the total ETH received without going through deposit or the fallback.

`WETH9.approve`

`WETH9.approve` preserves the S-06 balance relation. If, before the call, every account outside a finite list `S` has zero `balanceOf` and `∑_{a∈S} balanceOf a + forced = address(this).balance` (the value `totalSupply()` returns, with `forced` the ghost total of forced-in ETH), then on every successful run both facts still hold in the post-call state. `approve` writes only `allowance[msg.sender][guy]`, so `balanceOf` is

unchanged, and it moves no ETH.

`WETH9.«»`

The fallback `WETH9.«»` (which runs `deposit`) preserves S-06. Take a duplicate-free finite set S of accounts that contains `msg.sender`, with every account outside S at zero balance. Suppose that before the call `∑_{a∈S} balanceOf[a] + forced = address(this).balance − msg.value`. Then on success, `∑_{a∈S} balanceOf[a] + forced = address(this).balance` (the value `totalSupply()` returns), and every account outside S still has zero balance. The unchecked `balanceOf[msg.sender] += msg.value` cannot wrap under these hypotheses.

`WETH9.deposit`

S-06 is preserved by `WETH9.deposit`. Take a duplicate-free list `S` of addresses that contains `msg.sender` and covers every account with a nonzero balance. Suppose `∑_{a∈S} balanceOf a + forced` equals the contract's balance before `msg.value` arrived (`env.balance - env.value`). Then after a successful `deposit`, `∑_{a∈S} balanceOf a + forced = env.balance`, which is `address(this).balance`, the value `totalSupply()` returns. Every account outside `S` still holds zero. The unchecked `+=` cannot wrap: the sender's balance plus `msg.value` is at most the contract balance, which is below 2^256.

`WETH9.withdraw`

S-06 holds for `WETH9.withdraw`. The invariant is `∑_{a∈S} balanceOf[a] + forced = address(this).balance`, where `S` is a duplicate-free list of accounts containing `msg.sender`, every balance outside `S` is zero, and `forced` is the ghost total of ETH forced in.

If the invariant holds with the contract balance `env.balance` before a successful `withdraw(wad)`, then afterwards: * every account outside `S` still has zero balance; * `∑_{a∈S} balanceOf[a] + forced = env.balance − wad`, which is the contract's balance after `wad` wei go to `msg.sender`; * `wad ≤ env.balance`, so the subtraction never truncates.

The proof has three steps: * By the guard `balanceOf[msg.sender] ≥ wad`, the unchecked subtraction is exact. * Lowering one term of the sum by `wad` lowers the sum by exactly `wad` (lemma `S06.wd_sum_upd_sub`). * A call that reverts, or whose send fails, has no successful post-state.

`WETH9.transfer`

`WETH9.transfer` preserves the S-06 solvency relation. Suppose that before the call `S` is a list of accounts with no repeats, every `balanceOf` outside `S` is zero, and `∑_{a∈S} balanceOf a + forced = address(this).balance`, where `forced` is the total ETH forced into the contract. Then after any successful call there is a list `S'` with no repeats (`S` with `msg.sender` and `dst` added) such that every balance outside `S'` is zero and `∑_{a∈S'} balanceOf' a + forced = address(this).balance`. `transfer` is nonpayable and sends no ETH, so the contract's balance is the same before and after. The debit and credit cannot wrap, because the invariant bounds `balanceOf[sender] + balanceOf[dst]` by the

contract balance.

`WETH9.transferFrom`

`WETH9.transferFrom` preserves S-06. Let `S` be a duplicate-free list of accounts such that every `balanceOf` outside `S` is zero and `∑_{a∈S} balanceOf a + forced = address(this).balance`. Then after any successful `transferFrom(src, dst, wad)` there is a duplicate-free list `S'` such that every post-state `balanceOf` outside `S'` is zero and `∑_{a∈S'} balanceOf' a + forced = address(this).balance`. This holds on both allowance branches and when `src = dst`. The call is nonpayable, so the contract's ETH balance in the conclusion is the same value as in the hypothesis.

Assumptions. Forced ETH is modelled as ghost state: the running total of ETH that arrived without calling deposit or the fallback. withdraw's ETH transfer to msg.sender either succeeds in full or reverts the whole call (re-entry during the send leaves the relation intact). The contract starts with zero balance and zero balanceOf everywhere, apart from ETH already forced in, which counts toward the forced amount. The storage accessor satisfies `ext.get (ext.set w s) = s`. The contract's ETH balance is the call environment's `env.balance`, which the call cannot change (`approve` is nonpayable), and the forced-ETH ghost is fixed across the call. The sum over all addresses is written as a sum over a finite list `S`, with every account outside it holding a zero balance. `ext.get (ext.set w s) = s` (the storage round-trips). The ETH balance seen during the call already includes msg.value, and msg.value ≤ balance < 2^256. `forced` is ghost state. S is a duplicate-free list that contains the sender and covers every nonzero balance. `ext.get (ext.set w s) = s`. `env.value ≤ env.balance < 2^256`: the contract balance already includes `msg.value`. `forced` is a ghost counter for ETH forced into the contract. The sum over all addresses is represented by a finite duplicate-free support list `S` that contains the sender. - `ext.get (ext.set w s) = s`. - The sender's balance is below 2^256 (uint256 range). - The ETH send to `msg.sender` is opaque. When it succeeds, it is assumed to leave `balanceOf` exactly as it found it, meaning any re-entry nets out (the maintainer's re-entry assumption). - The contract's balance after the call is taken to be `env.balance − wad`, i.e. the send moves exactly `wad` wei out. The storage accessor is lawful (`get (set w s) = s`, `set (set w s) t = set w t`). the contract's ETH balance is below 2^256. the pre-state list `S` has no repeats. The storage lens `ext` is lawful: get-after-set returns the value set, and a second set overwrites the first. `address(this).balance < 2^256`. `forced` is ghost state that a nonpayable call leaves unchanged.

5. Scope and limits

- The proofs are about the Lean model formal-ast generates from solc's AST. They assume the compiler preserves the source's meaning; the bytecode itself isn't verified.
- Gas is not modelled. `withdraw` sends ETH with `transfer` (2300 gas): a recipient that rejects it, or needs more gas, can't withdraw. S-05 states this exception.
- ETH that arrives without a call (selfdestruct, block rewards) is modelled as an increase in the contract's balance; S-02 and S-06 account for it.
- Statements about sequences of calls (S-05) are proved one function at a time: no call by another account to any function breaks the claim.
- `name`, `symbol` and `decimals` are constants and outside the proofs.

Exported from the proof ledger and `.formal/statements.md` at `2cd3283`.